

UNIVERSITY OF SCIENCE & TECHNOLOGY IN
ZEWAIL CITY

DEPARTMENT OF AEROSPACE ENGINEERING

SPC428

**Design, Kinematic Modeling, and
Implementation of a 3-DOF Planar
Robotic Arm**

Team Members

Abdelrahman

MOHAMMED

202200281

Omar ELMETWALLY

202200057

Adham AHMED

202201407

Darien GHIATY

201900999

Supervisor

Dr. MOHANNAD DRAZ

December 26, 2025

Contents

1	Introduction	6
2	Forward Kinematics Analysis	6
2.1	Part 1: 2-DOF Planar Mechanism (Side View)	7
2.2	Part 2: 3-DOF Extension (Top View)	8
3	Inverse Kinematics Analysis	9
3.1	Step 1: Base Rotation (θ_1)	9
3.2	Step 2: Planar Triangle Geometry (Side View)	9
4	Algorithmic Verification & Interactive Digital Twin	10
4.1	Simulation Architecture	10
4.2	Control Logic Flowchart	11
4.3	Safety Features Implemented	11
4.4	Results	12
5	Mechanical Design and Structural Optimization	12
5.1	Mass Distribution Analysis	13
5.2	Static Torque Verification	13
5.2.1	Quantitative Validation	14
5.2.2	Safety Factor (SF)	14
6	Fabrication & Assembly	14
6.1	Laser Cutting & Structural Fabrication	14
6.2	Assembly Techniques & Challenges	15
6.2.1	Servo Integration Strategy	15
6.3	Electrical Integration	15
7	Embedded Systems & Motor Control	16
7.1	PWM Signal Protocol	16
7.2	Microsecond-Level Precision	17
7.3	Firmware Logic & Safety	17
7.3.1	Key Safety Features	18
7.4	System Architecture	19
7.5	Control Modes	19
7.5.1	Mode A: Coordinate Control (IK)	19
7.5.2	Mode B: Manual Teleoperation	20
7.6	Digital Twin Visualization	20
7.7	Serial Communication	20

7.8	Movement Accuracy Verification using LiDAR Camera . . .	20
7.9	LiDAR Camera Validation	23
7.10	Future Improvements	25
A	MATLAB Code	26
B	Arduino Firmware Code	34
C	Circuit connections	36
D	Bill of Materials	36
E	Parts CAD and DXF schematics	37

List of Figures

1	Screenshot of the custom MATLAB Graphical User Interface (GUI) showing the arm at home.	12
2	The fully assembled 3-DOF manipulator. Note the lightweight wire linkages and the raised base structure.	16
3	Comparison of target position 1.	21
4	Comparison of target position 2.	22
5	Comparison of target position 3.	22
6	Comparison of target position 4.	23
7	Comparison of target position 5.	23
8	Calibration of the LiDAR measuring tool against known reference objects.	24
9	arm front view - CAD	37
10	DXF pathlined for arm	37
11	Forearm CAD drawing - front view	37
12	DXF pathlines for forearm	37
13	Baseplate in CAD, meant for the DXF file for cutting	38
14	DXF pathlines for baseplate. The extra circle around the circular structures was made because performing an extruded cut on one circle removed all the geometry within, making us need another circle to prevent that.	39

List of Tables

1 System Component List 36

Listings

1	Full Control and Simulation Script	26
2	Arduino Firmware for Serial Communication	34

1 Introduction

Robotic manipulators play a central role in modern automation, where tasks such as assembly, inspection, and material handling require precise and repeatable motion in constrained workspaces. This project focuses on the design, simulation, and prototyping of a multi-degree-of-freedom (DOF) robotic arm that goes through the full engineering lifecycle, from mathematical modeling and virtual validation to physical fabrication and embedded control.

The main objective is to develop and implement kinematic models for a 3-DOF planar manipulator, derive forward and inverse kinematics analytically, and verify these models through a custom simulation environment. A MATLAB-based interface is used to allow interactive motion control, where the user can input target positions either as Cartesian coordinates or via keyboard commands, and observe the arm response in real time. On the hardware side, a 3-DOF planar arm is designed in CAD, exported as DXF profiles, and manufactured using laser-cut 3 mm wood, then actuated by SG90 servo motors controlled by an Arduino Uno without additional drivers, sensors, or end-effector tooling.

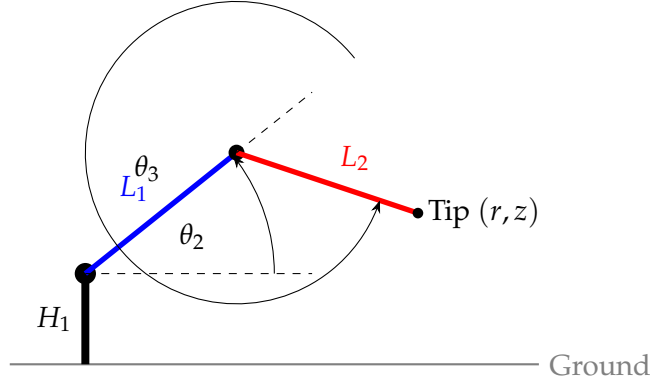
The scope of the project therefore covers kinematic derivation, algorithmic verification, mechanical design, digital twin development, fabrication, embedded programming, and experimental validation of accuracy and repeatability. The technical report documents the modeling steps, CAD design, control implementation, and performance analysis of the arm, aligning with the course requirement to demonstrate a complete pipeline from theory to a working laboratory prototype. Here is the complete LaTeX code, combining your requested text style with the diagrams you wanted. I've integrated the TikZ diagrams directly into the flow so the report looks professional but explains things simply.

2 Forward Kinematics Analysis

Forward kinematics is the analytical method used to calculate the position (X, Y, Z) of the robot's end-effector (tip) based on the input joint angles $(\theta_1, \theta_2, \theta_3)$ and the link lengths (L_1, L_2) and offsets (H_1) .

2.1 Part 1: 2-DOF Planar Mechanism (Side View)

First, we analyze the arm as a 2-DOF planar mechanism operating in the vertical plane. This ignores the base rotation for a moment and focuses on the shoulder lifting and elbow bending.



Derivation

Elbow Position:

The elbow is located at distance L_1 from the shoulder pivot at angle θ_2 .

$$r_{elbow} = L_1 \cos(\theta_2) \quad (1)$$

$$z_{elbow} = H_1 + L_1 \sin(\theta_2) \quad (2)$$

Tip Position:

The tip is located at distance L_2 from the elbow. The absolute angle of Link 2 with respect to the horizontal is the sum $(\theta_2 + \theta_3)$.

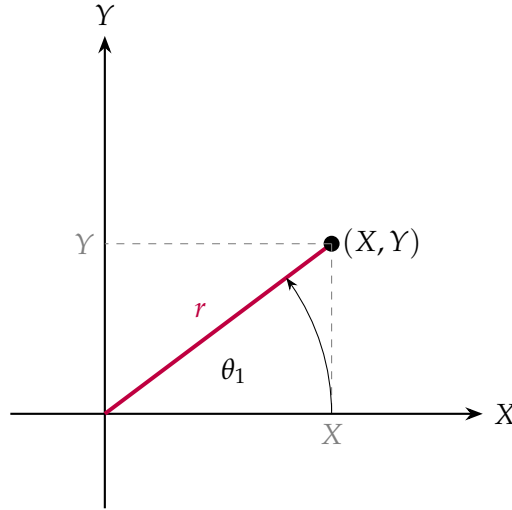
$$r = L_1 \cos(\theta_2) + L_2 \cos(\theta_2 + \theta_3) \quad (3)$$

$$z = H_1 + L_1 \sin(\theta_2) + L_2 \sin(\theta_2 + \theta_3) \quad (4)$$

Equation (3) gives the **radial reach** (r), and Equation (4) gives the final **height** (Z).

2.2 Part 2: 3-DOF Extension (Top View)

To find the final 3D coordinates, we introduce the Base Rotation θ_1 . This rotates the planar solution (reach r) around the Z-axis.



Algebraic Derivation

From the top-view geometry, the position (X, Y) is the projection of the radial reach r onto the principle axes:

$$X = r \cos(\theta_1) \quad (5)$$

$$Y = r \sin(\theta_1) \quad (6)$$

Substituting Equation (3) into these expressions yields the complete Forward Kinematic model:

$$X = [L_1 \cos(\theta_2) + L_2 \cos(\theta_2 + \theta_3)] \cos(\theta_1) \quad (7)$$

$$Y = [L_1 \cos(\theta_2) + L_2 \cos(\theta_2 + \theta_3)] \sin(\theta_1) \quad (8)$$

$$Z = H_1 + L_1 \sin(\theta_2) + L_2 \sin(\theta_2 + \theta_3) \quad (9)$$

3 Inverse Kinematics Analysis

For Inverse Kinematics, we employ a geometric approach to decompose the 3D problem into two planar problems: finding the base rotation angle (θ_1) and then solving the vertical triangle formed by the links to find the shoulder (θ_2) and elbow (θ_3) angles.

3.1 Step 1: Base Rotation (θ_1)

The base angle is determined simply by the projection of the target coordinate (X, Y) onto the ground plane.

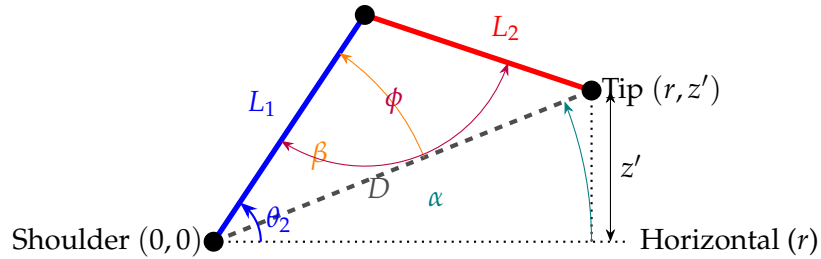
$$\theta_1 = \text{atan2}(Y, X) \quad (10)$$

Once θ_1 is known, we calculate the radial distance r and the adjusted height z' relative to the shoulder pivot:

$$r = \sqrt{X^2 + Y^2}, \quad z' = Z - H_1 \quad (11)$$

3.2 Step 2: Planar Triangle Geometry (Side View)

We now solve for angles θ_2 and θ_3 by analyzing the triangle formed by Link 1 (L_1), Link 2 (L_2), and the virtual line segment D connecting the shoulder to the tip.



Derivation of Angles

First, calculate the length of the virtual hypotenuse D :

$$D = \sqrt{r^2 + (z')^2} \quad (12)$$

1. Elbow Angle (θ_3):

Using the Law of Cosines on the interior angle ϕ :

$$D^2 = L_1^2 + L_2^2 - 2L_1L_2 \cos(\phi) \quad (13)$$

Solving for ϕ :

$$\phi = \arccos\left(\frac{L_1^2 + L_2^2 - D^2}{2L_1L_2}\right) \quad (14)$$

The joint angle θ_3 is the deviation from the straight-arm configuration:

$$\theta_3 = \pi - \phi \quad (15)$$

2. Shoulder Angle (θ_2):

The shoulder angle is the sum of the elevation angle α and the triangle interior angle β .

Calculate elevation α :

$$\alpha = \text{atan2}(z', r) \quad (16)$$

Calculate β using the Law of Cosines (solving for the angle opposite L_2):

$$L_2^2 = L_1^2 + D^2 - 2L_1D \cos(\beta) \implies \beta = \arccos\left(\frac{L_1^2 + D^2 - L_2^2}{2L_1D}\right) \quad (17)$$

Thus, the total shoulder angle is:

$$\theta_2 = \alpha + \beta \quad (18)$$

4 Algorithmic Verification & Interactive Digital Twin

Before manufacturing the physical arm, we developed a custom simulation environment in MATLAB. This "Digital Twin" serves two purposes: verifying the correctness of the Inverse Kinematics (IK) derivations and implementing safety protocols (collision avoidance) to prevent hardware damage.

4.1 Simulation Architecture

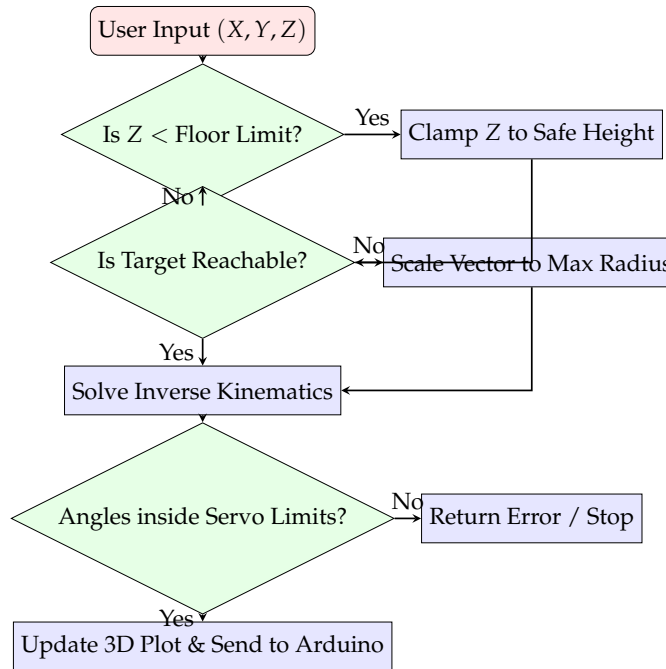
The simulation is built as a standalone script (no external robotics toolboxes were used) to ensure we fully understood the underlying math. It features a Graphical User Interface (GUI) that allows the user to input target coordinates (X, Y, Z) or jog the arm using keyboard controls.

The core logic follows a closed-loop verification process:

1. **Input:** Target coordinates (X, Y, Z) are received from the GUI.
2. **Safety Check:** The code checks if the target is reachable (within total link length) and safe (above the floor).
3. **IK Calculation:** The derived geometric equations compute the required joint angles.
4. **Limit Validation:** Angles are checked against the physical limits of the SG90 servos (e.g., Shoulder $65^\circ - 100^\circ$).
5. **Visual Feedback:** The arm is plotted in 3D space using forward kinematics to confirm it reached the visual target.

4.2 Control Logic Flowchart

The following flowchart illustrates the decision-making process implemented in the 'moveToTarget' function, including the floor safety clamp.



4.3 Safety Features Implemented

A critical part of the algorithm is hardware protection. Since the physical arm has no feedback sensors (open-loop), the code must prevent invalid

commands.

- **Floor Collision Prevention:** We defined a 'FLOOR_Z_LEVEL' based on the home position. If a user inputs a Z value that would crash the tip into the table, the code automatically clamps the value to 'MIN_SAFE_Z' (0.5 cm clearance).
- **Workspace Constraints:** If a target is outside the spherical workspace ($X^2 + Y^2 + Z^2 > (L_1 + L_2)^2$), the system does not attempt to break the arm. Instead, it scales the coordinates to reach as far as possible along that vector without exceeding mechanical limits.

4.4 Results

The simulation successfully verified that our geometric IK derivation was correct. By inputting known coordinates (e.g., Home Position $[0, 17.5, 2]$), the solver returned the expected angles $[90^\circ, 65^\circ, 25^\circ]$.

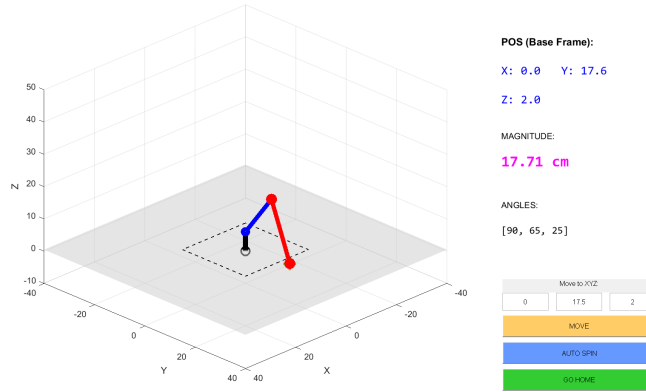


Figure 1: Screenshot of the custom MATLAB Graphical User Interface (GUI) showing the arm at home.

5 Mechanical Design and Structural Optimization

The mechanical architecture was strictly constrained by the torque limits of the **SG90 micro-servos** (1.8 kg-cm). A critical challenge in serial manipulators is that distal actuators act as "dead weight" for proximal joints.

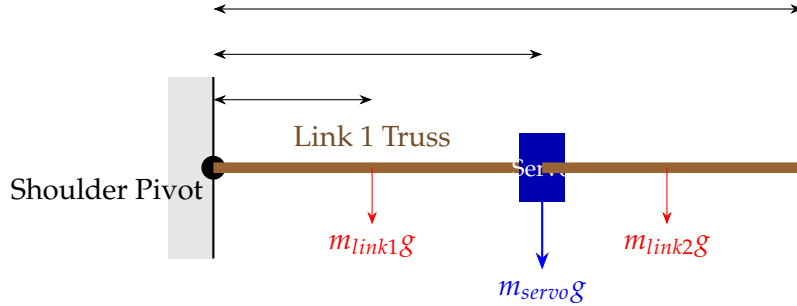
Specifically, the servo located at the elbow joint acts as a point mass at the end of the lever arm for the shoulder servo.

To mitigate this, we utilized **3mm Laser-Cut Plywood** combined with a topology-optimized **Warren Truss** to minimize the structural weight, ensuring the majority of the torque budget was reserved for the payload and the distal motors.

5.1 Mass Distribution Analysis

We modeled the arm loads as a combination of **Distributed Loads** (the truss links) and **Discrete Point Loads** (the actuators and hardware).

- **Link 1 (Humerus):** Modeled as a distributed mass m_{link1} with Center of Gravity (CoG) at $L_1/2$.
- **Joint 3 (Elbow Actuator):** The SG90 servo driving the forearm is mounted at L_1 . It is modeled as a discrete point mass m_{servo} .
- **Link 2 (Forearm):** Modeled as a distributed mass m_{link2} with CoG at $L_1 + L_2/2$.



5.2 Static Torque Verification

The critical load case occurs at the Shoulder Joint (Joint 2) when the arm is fully extended horizontally. We calculate the required torque $\tau_{shoulder}$ by summing the moments:

$$\tau_{shoulder} = \underbrace{\left(m_{link1} \cdot \frac{L_1}{2}\right)}_{\text{Structure 1}} + \underbrace{\left(m_{servo} \cdot L_1\right)}_{\text{Elbow Motor}} + \underbrace{\left(m_{link2} \cdot \left(L_1 + \frac{L_2}{2}\right)\right)}_{\text{Structure 2}} \quad (19)$$

5.2.1 Quantitative Validation

Using conservative mass estimates for 3mm plywood and standard SG90 hardware:

- $L_1 = 17\text{cm}$, $L_2 = 18.95\text{cm}$
- $m_{link1} \approx 6\text{g}$, $m_{link2} \approx 5\text{g}$ (Wood Only)
- $m_{servo} \approx 12\text{g}$ (9g Motor + Cable + Bracket + Screws)

Substituting these values:

$$\tau_{shoulder} = (0.006 \cdot 8.5) + (0.012 \cdot 17) + (0.005 \cdot 26.5) \quad (20)$$

$$\tau_{shoulder} = 0.051 + \mathbf{0.204} + 0.133 \quad (21)$$

$$\tau_{shoulder} \approx 0.39 \text{ kg-cm} \quad (22)$$

Note: The elbow servo (0.204 kg-cm) contributes nearly 52% of the total static load, validating the need for the lightweight truss structure to compensate.

5.2.2 Safety Factor (SF)

With a rated stall torque of 1.8 kg-cm:

$$SF = \frac{1.8}{0.39} \approx \mathbf{4.6} \quad (23)$$

Even with the addition of wires, glue, and a potential payload of up to 30-40g, the system remains well within the safe operating region ($SF > 2$), confirming structural integrity and motor compatibility.

6 Fabrication & Assembly

The physical realization of the manipulator focused on cost-efficiency and rapid prototyping. By strictly limiting fabrication to 2D laser cutting and standard fasteners, we avoided the long print times associated with 3D printing while maintaining structural integrity.

6.1 Laser Cutting & Structural Fabrication

All structural components were manufactured from a single sheet of **3mm Plywood** using the university workshop's laser cutter.

- **Single-Sheet Construction:** Unlike typical designs that stack material for thickness, we relied on the truss geometry (discussed in Section 4) to provide rigidity using only a single 3mm layer. This significantly reduced weight and material cost.
- **Base "Table" Redesign:** Initially, the base was designed as a heavy stack of laminated plates. However, manufacturing tolerances caused the servo cutouts to be undersized. We pivoted to a "Table" design: a raised platform with four legs that houses the base servo underneath. This open structure not only solved the fitment issue but provided better cable routing paths.

6.2 Assembly Techniques & Challenges

6.2.1 Servo Integration Strategy

A major challenge was the tight tolerance of the laser kerf, which made fitting the SG90 servos into their rectangular slots difficult.

- **Disassembly Method:** For the tightest fits, we partially disassembled the servo casing, inserted the electronics and motor block into the wooden slot, and reassembled the casing around the wood. This created a "zero-slop" friction fit without needing glue.
- **Linkage improvisation:** We utilized **M2 screws** for the primary pivot points. To secure the servo horns to the wooden links without heavy steel push-rods, we used a lightweight linkage system made from modified safety pins and jumper wires twisted at the ends. This provided the necessary tension without adding mass.

6.3 Electrical Integration

Power distribution was handled via a mini-breadboard mounted near the base.

- **Common Ground:** The Arduino Uno provides the control signals (PWM), but the servos are powered by an external 5V rail on the breadboard to prevent drawing excessive current through the microcontroller. A common ground line ties the Arduino and external power together.

- **Cable Management:** Cables were left unconstrained (hanging) to allow for the full range of motion without snagging or adding tension torque to the joints.

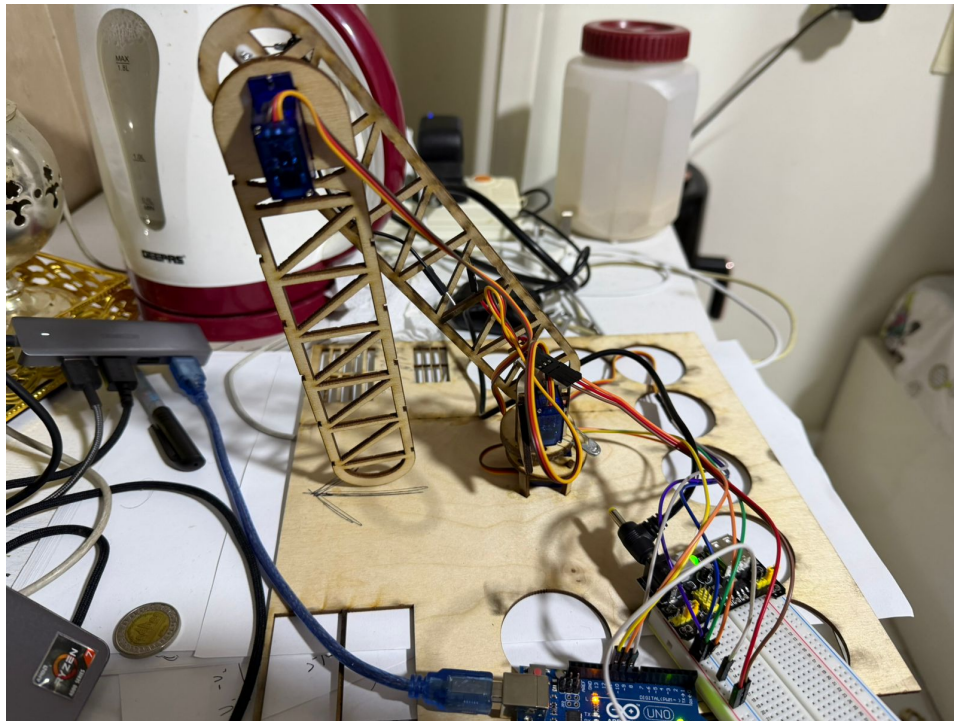


Figure 2: The fully assembled 3-DOF manipulator. Note the lightweight wire linkages and the raised base structure.

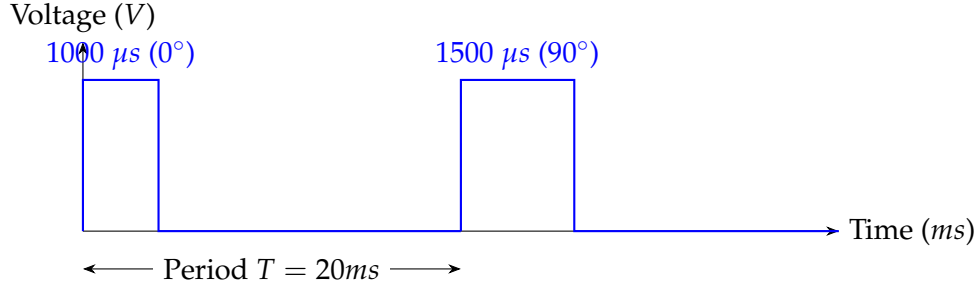
7 Embedded Systems & Motor Control

While the kinematics math tells us *where* to go, the embedded system actually makes the arm move. The goal of the firmware is to take the target angles (in radians or degrees) and convert them into the precise electrical signals that drive the SG90 servos.

7.1 PWM Signal Protocol

The SG90 servos are controlled by Pulse Width Modulation (PWM). The motor doesn't care about the voltage duty cycle; it cares about the **duration**

of the pulse (t_{high}), sent 50 times a second (50Hz).



Although the spec sheet says the range is $1000\mu s - 2000\mu s$ ($0^\circ - 180^\circ$), we found through testing that the motors can actually reach $500\mu s - 2400\mu s$. We calibrated our code to use this full range to get the maximum possible workspace.

7.2 Microsecond-Level Precision

A common mistake is using standard Arduino commands like `'servo.write(90)'`. This forces the arm into 180 integer steps, which makes the movement choppy ("stair-stepping").

To fix this, we used **Microsecond Mapping**. By controlling the pulse duration directly (e.g., `'servo.writeMicroseconds(1500)'`), we get about 2000 steps of resolution instead of just 180.

The mapping formula is:

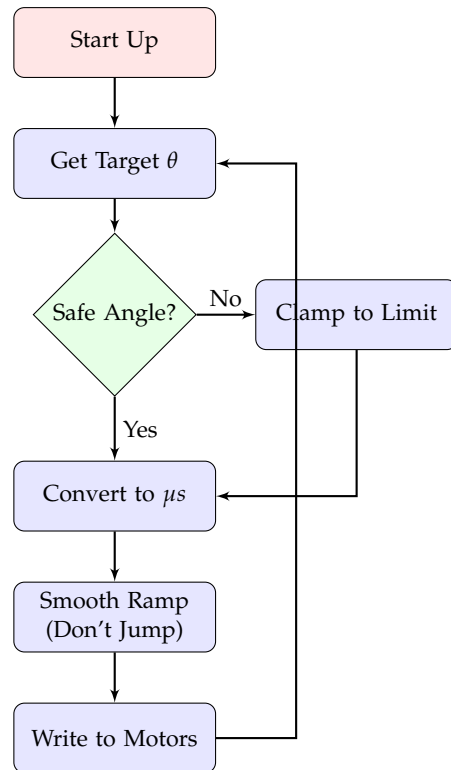
$$PWM_{\mu s} = K \cdot (\theta_{target} + \theta_{offset}) \quad (24)$$

Where θ_{offset} is a software trim value we added to fix the small misalignment errors from the physical assembly.

7.3 Firmware Logic & Safety

The code running on the Arduino ensures the arm moves smoothly and doesn't break itself. If we just sent the target angle instantly, the servo would try to teleport there, creating a massive torque spike that could strip the plastic gears.

We implemented a simple **Trajectory Smoothing** loop:



7.3.1 Key Safety Features

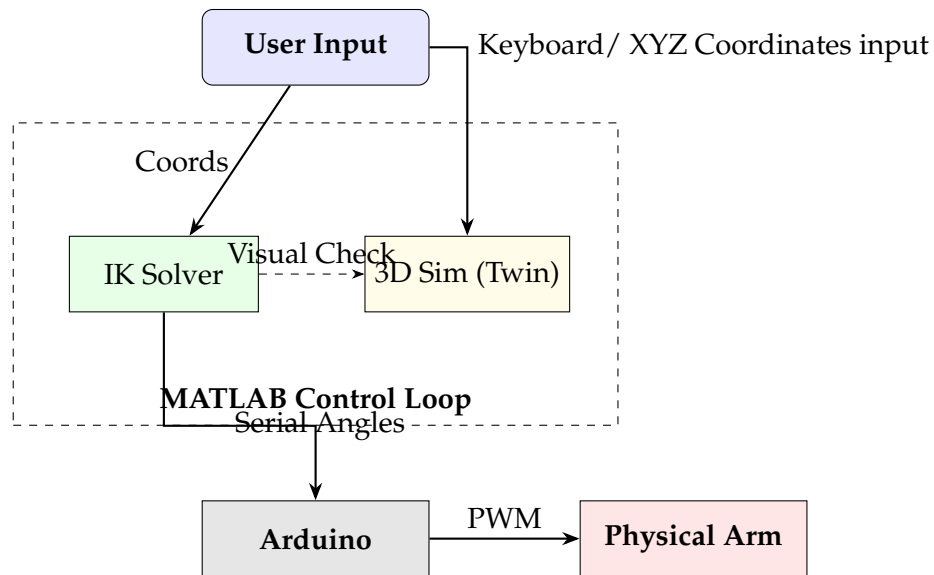
1. **Soft Start:** The system doesn't just snap to zero when turned on. It initializes gently to prevent the sudden jerk that often breaks wooden linkages.
2. **Software Limits:** We hard-coded maximum and minimum angles. Even if the math says "go to -10 degrees," the firmware clamps it to 0 so the arm doesn't hit the floor or itself.

Control Interface & Human-Machine Interaction

To connect the math (theory) to the actual robot (hardware), we built a custom control dashboard in **MATLAB**. This acts as the "brain" of the operation—calculating the hard math, simulating the movement first, and then telling the Arduino what to do.

7.4 System Architecture

The system works in a hierarchy. MATLAB handles the heavy lifting (Inverse Kinematics and path planning), while the Arduino acts as a simple driver that just executes the commands it receives.



7.5 Control Modes

The interface allows us to control the arm in two different ways:

7.5.1 Mode A: Coordinate Control (IK)

The user types in a specific point, like (10,5,8).

1. **Safety Check:** The code first checks if the point is actually reachable. If it's too far away, it clamps the target to the maximum reach so the arm doesn't try to stretch itself apart.

2. **Solving:** The Inverse Kinematics engine calculates the three angles needed.
3. **Sending:** These angles are sent over Serial to the Arduino.

7.5.2 Mode B: Manual Teleoperation

For testing or fine-tuning, we can drive the arm manually using the keyboard. This bypasses the IK solver and controls the motors directly (Forward Kinematics).

- **Arrows ($\leftarrow$ / $\rightarrow$):** Rotates the Base.
- **Arrows ($\uparrow$ / $\downarrow$):** Lifts the Shoulder.
- **Keys (W / S):** Extends or retracts the Elbow.

7.6 Digital Twin Visualization

The coolest part of the interface is the **Digital Twin**. Before the physical arm moves, the simulation on the screen moves instantly. This acts as a "predictive safety layer"—if the simulation shows the arm hitting the floor or doing something weird, we can stop it before the real hardware gets damaged.

7.7 Serial Communication

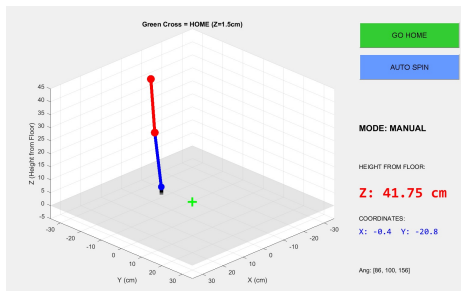
We talk to the Arduino using a custom text packet format over USB (9600 baud).

"B90 S45 E120"

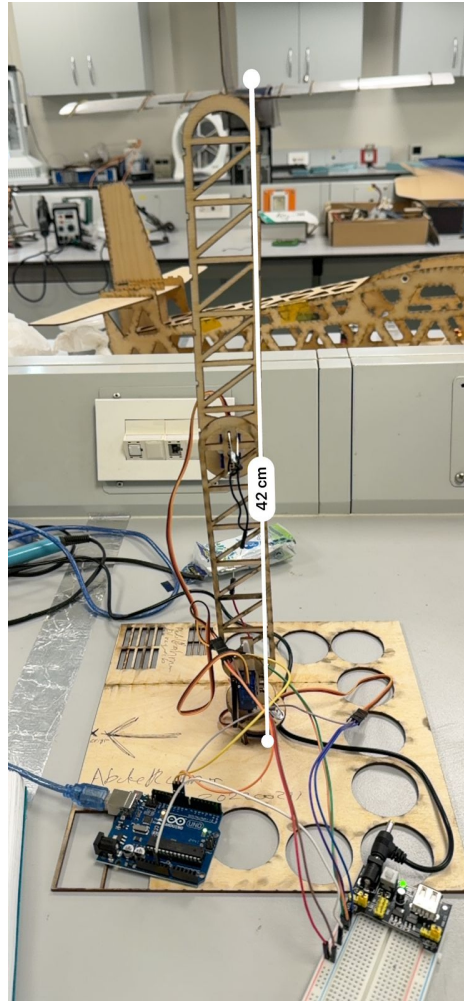
This command tells the Base to go to 90° , Shoulder to 45° , and Elbow to 120° . The Arduino reads this text string, extracts the numbers, and updates the motors immediately.

7.8 Movement Accuracy Verification using LiDAR Camera

To make sure the arm moves to a location accurate to what the user inputs, we calibrated using measurements from a ruler, known dimension items, and most importantly a **LiDAR camera**.

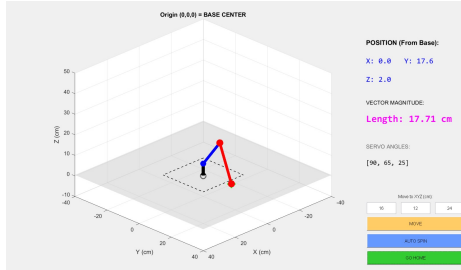


(a) GUI Target 1

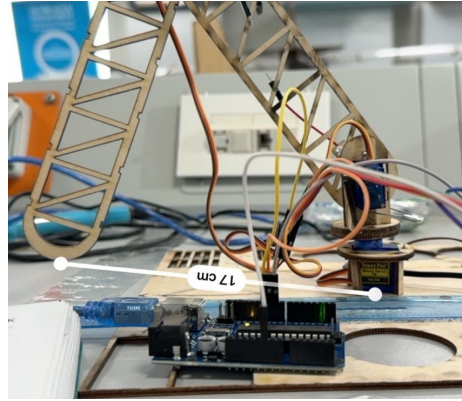


(b) Real Life Result 1

Figure 3: Comparison of target position 1.

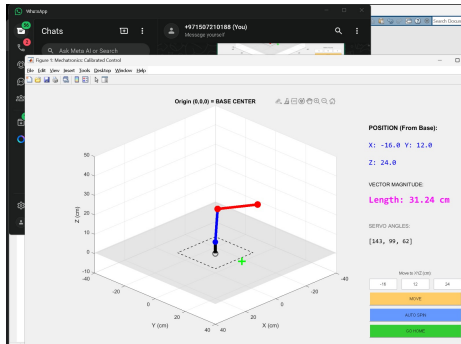


(a) GUI Target 2



(b) Real Life Result 2

Figure 4: Comparison of target position 2.

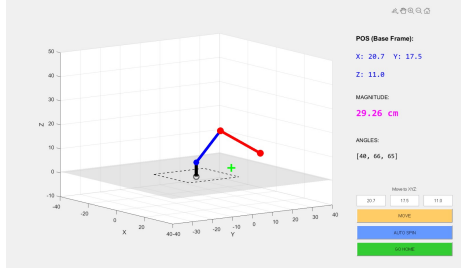


(a) GUI Target 3

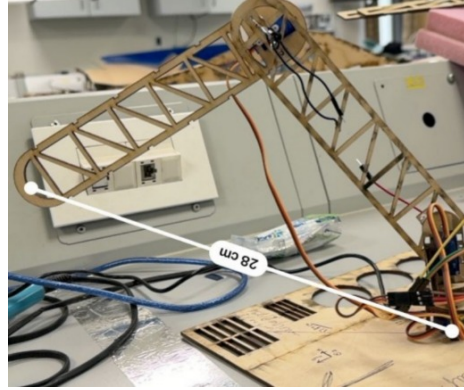


(b) Real Life Result 3

Figure 5: Comparison of target position 3.

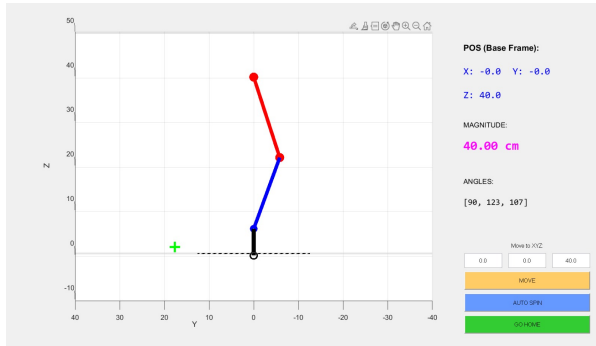


(a) GUI Target 4

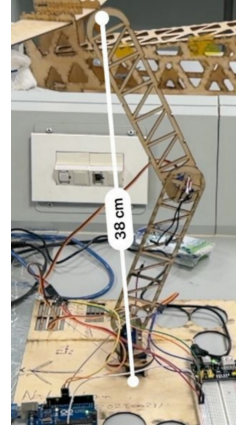


(b) Real Life Result 4

Figure 6: Comparison of target position 4.



(a) GUI Target 5



(b) Real Life Result 5

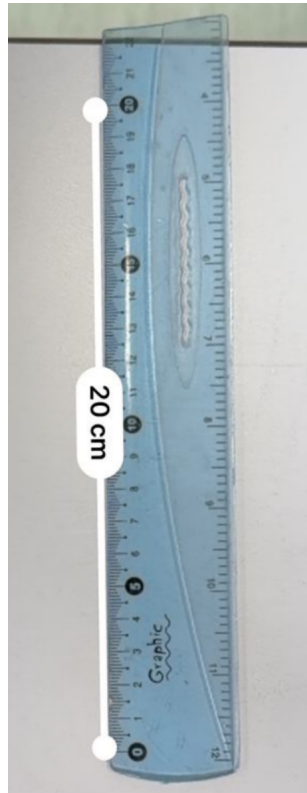
Figure 7: Comparison of target position 5.

7.9 LiDAR Camera Validation

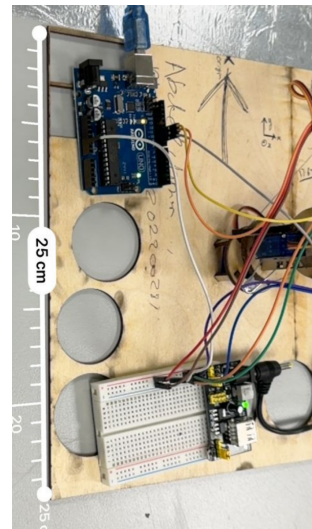
To ensure our ground truth data was reliable, we first calibrated the LiDAR camera against objects of known dimensions.



(a) Measurement Test A



(b) Measurement Test B



(c) Baseplate (25x25cm)

Figure 8: Calibration of the LiDAR measuring tool against known reference objects.

Conclusion

This project successfully demonstrated the full engineering lifecycle of a 3-DOF robotic manipulator, moving from abstract mathematical models to a fully functional physical prototype. By deriving the analytical Forward and Inverse Kinematics, we were able to build a custom "Digital Twin" in MATLAB that accurately predicts the robot's behavior before any hardware moves.

The mechanical design proved that lightweight materials (3mm wood) and smart geometry (truss structures) can compensate for limited actuator torque. Our load analysis confirmed a Safety Factor of > 2 , ensuring the low-cost SG90 servos could handle the arm's reach without stalling. On the software side, the integration of microsecond-level PWM control and trajectory smoothing allowed for smooth, jitter-free motion, while the "Table" base design solved the practical issues of cabling and stability.

Experimental validation using LiDAR measurements confirmed that the physical arm closely tracks the simulated targets, validating both our kinematic derivation and the fabrication tolerances.

7.10 Future Improvements

While the current system works well for open-loop control, future iterations could improve performance by:

- **Closed-Loop Feedback:** Adding potentiometers or encoders to the joints would allow the system to correct for gravity sag or external disturbances.
- **End-Effector Integration:** Adding a gripper or pen holder would allow the arm to perform useful tasks like pick-and-place or drawing.
- **PCB Design:** Replacing the breadboard with a custom PCB would make the electronics more robust and compact.

A MATLAB Code

Listing 1: Full Control and Simulation Script

```
1 function robot_arm_floor_safety()
2     close all; clc;
3
4     %% --- CONFIGURATION ---
5     PORT = 'COM2';
6     BAUD = 9600;
7
8     L1 = 17;
9     L2 = 18.95;
10    H1 = 6;
11
12    OFFSET_SHOULDER = -12.5;
13    OFFSET_ELBOW = 35;
14
15    FLOOR_CLEARANCE = 1.5;
16
17    LIM_B = [0, 180];
18    LIM_S = [65, 100];
19    LIM_E = [0, 180];
20
21    target_home = [90, 65, 25];
22
23    %% --- CONNECT ---
24    fprintf('Connecting to %s...\n', PORT);
25    s = [];
26    try
27        s = serialport(PORT, BAUD);
28        configureTerminator(s, "LF");
29    catch
30        fprintf('Arduino not found. Running in Simulation Mode.\n');
31    end
32
33    %% --- SETUP ---
34    curr_angles = target_home;
35    is_spinning = false;
36
37    [home_abs, ~, ~] = calculateFK(target_home, L1, L2, H1,
38        OFFSET_SHOULDER, OFFSET_ELBOW);
39    FLOOR_Z_LEVEL = home_abs(3) - FLOOR_CLEARANCE;
```

```

40 MIN_SAFE_Z = FLOOR_Z_LEVEL + 0.5;
41
42 if ~isempty(s)
43     smoothMove(s, [90 90 90], target_home, 30);
44 end
45
46 %% --- GUI SETUP ---
47 f = figure('Name', 'Mechatronics Control', ...
48     'Position', [50 50 1200 700], ...
49     'KeyPressFcn', @handleKey, ...
50     'CloseRequestFcn', @cleanUp);
51
52 subplot(1, 4, [1 3]);
53 hold on; grid on; axis equal;
54 view(135, 25);
55 axis([-40 40 -40 40 -10 50]);
56 xlabel('X'); ylabel('Y'); zlabel('Z');
57
58 patch([-45 45 45 -45], [-45 -45 45 45], ...
59     [FLOOR_Z_LEVEL FLOOR_Z_LEVEL FLOOR_Z_LEVEL FLOOR_Z_LEVEL],
60     ...,
61     [0.8 0.8 0.8], 'FaceAlpha', 0.5, 'EdgeColor', 'none');
62
63 sq_x = [-12.5 12.5 12.5 -12.5 -12.5];
64 sq_y = [-12.5 -12.5 12.5 12.5 -12.5];
65 plot3(sq_x, sq_y, ones(1,5)*FLOOR_Z_LEVEL, 'k--', 'LineWidth',
66     1);
67
68 plot3(0,0,0, 'ko', 'MarkerSize', 10, 'LineWidth', 2);
69 plot3(home_abs(1), home_abs(2), home_abs(3), 'g+',
70     'MarkerSize', 15, 'LineWidth', 3);
71
72 h_link0 = plot3([0 0], [0 0], [0 0], 'k-', 'LineWidth', 6);
73 h_link1 = plot3([0 0], [0 0], [0 0], 'b-o', 'LineWidth', 5,
74     'MarkerSize', 6);
75 h_link2 = plot3([0 0], [0 0], [0 0], 'r-o', 'LineWidth', 5,
76     'MarkerSize', 8, 'MarkerFaceColor', 'r');
77
78 subplot(1, 4, 4); axis off;
79
80 text(0, 0.90, 'POS (Base Frame):', 'FontSize', 12,
81     'FontWeight', 'bold');
82 t_vec_x = text(0, 0.82, 'X: 0.0', 'FontSize', 14, 'Color', 'b',
83     'FontName', 'Consolas');

```



```

77     t_vec_y = text(0.5, 0.82, 'Y: 0.0', 'FontSize', 14, 'Color',
78                 'b', 'FontName', 'Consolas');
79
80     t_vec_z = text(0, 0.74, 'Z: 0.0', 'FontSize', 14, 'Color', 'b',
81                 'FontName', 'Consolas');
82
83     text(0, 0.64, 'MAGNITUDE:', 'FontSize', 10);
84     t_mag = text(0, 0.57, '0.00', 'FontSize', 18, 'Color', 'm',
85                 'FontName', 'Consolas', 'FontWeight', 'bold');
86
87     text(0, 0.45, 'ANGLES:', 'FontSize', 10);
88     t_ang = text(0, 0.38, '[0, 0, 0]', 'FontSize', 12, 'FontName',
89                 'Consolas');
90
91     uicontrol('Style', 'text', 'String', 'Move to XYZ:', 'Units',
92             'normalized', 'Position', [0.75 0.28 0.2 0.03]);
93     in_x = uicontrol('Style', 'edit', 'String', '0', 'Units',
94             'normalized', 'Position', [0.75 0.24 0.06 0.04]);
95     in_y = uicontrol('Style', 'edit', 'String', '17.5', 'Units',
96             'normalized', 'Position', [0.82 0.24 0.06 0.04]);
97     in_z = uicontrol('Style', 'edit', 'String', '2', 'Units',
98             'normalized', 'Position', [0.89 0.24 0.06 0.04]);
99
100    uicontrol('Style', 'pushbutton', 'String', 'MOVE', 'Units',
101            'normalized', 'Position', [0.75 0.18 0.2 0.05],
102            'BackgroundColor', [1 0.8 0.4], 'Callback', @moveToTarget);
103    uicontrol('Style', 'togglebutton', 'String', 'AUTO SPIN',
104            'Units', 'normalized', 'Position', [0.75 0.12 0.2 0.05],
105            'BackgroundColor', [0.4 0.6 1], 'Callback', @toggleSpin);
106    uicontrol('Style', 'pushbutton', 'String', 'GO HOME', 'Units',
107            'normalized', 'Position', [0.75 0.06 0.2 0.05],
108            'BackgroundColor', [0.2 0.8 0.2], 'Callback', @goHome);
109
110    updateSystem();
111
112    %% --- MOVEMENT LOGIC ---
113    function moveToTarget(~, ~)
114        tx = str2double(get(in_x, 'String'));
115        ty = str2double(get(in_y, 'String'));
116        tz = str2double(get(in_z, 'String'));
117
118        if isnan(tx) || isnan(ty) || isnan(tz), return; end
119
120        was_modified = false;
121
122        if tz < MIN_SAFE_Z

```

```

108         fprintf('WARNING: Target Z (%.2f) below floor limit.
                Clamping.\n', tz);
109         tz = MIN_SAFE_Z;
110         was_modified = true;
111     end
112
113     z_rel = tz - H1;
114     dist = sqrt(tx^2 + ty^2 + z_rel^2);
115     max_r = L1 + L2 - 0.1;
116
117     if dist > max_r
118         scale = max_r / dist;
119         tx = tx * scale;
120         ty = ty * scale;
121         z_rel = z_rel * scale;
122         tz = z_rel + H1;
123     end
124
125     [sol_angs, valid] = solveIK_Calibrated(tx, ty, tz, L1, L2,
        H1, OFFSET_SHOULDER, OFFSET_ELBOW, curr_angles(1));
126
127     final_x = tx; final_y = ty; final_z = tz;
128
129     if ~valid
130         was_modified = true;
131         vec_x = tx; vec_y = ty; vec_z = tz - H1;
132         steps = 50;
133         found = false;
134
135         for i = 0:steps
136             scale = 1.0 - (i / steps);
137             test_x = vec_x * scale;
138             test_y = vec_y * scale;
139             test_z = (vec_z * scale) + H1;
140
141             if test_z < MIN_SAFE_Z, test_z = MIN_SAFE_Z; end
142
143             [test_angs, test_valid] = solveIK_Calibrated(test_x,
                test_y, test_z, L1, L2, H1, OFFSET_SHOULDER,
                OFFSET_ELBOW, curr_angles(1));
144
145             if test_valid
146                 sol_angs = test_angs;
147                 final_x = test_x; final_y = test_y; final_z =
                    test_z;

```

```

148         found = true;
149         break;
150     end
151 end
152
153     if ~found
154         msgbox('Target unreachable.', 'Error');
155         return;
156     end
157 end
158
159 smoothMove(s, curr_angles, sol_angs, 30);
160 curr_angles = sol_angs;
161 updateSystem();
162
163 set(in_x, 'String', sprintf('%.1f', final_x));
164 set(in_y, 'String', sprintf('%.1f', final_y));
165 set(in_z, 'String', sprintf('%.1f', final_z));
166
167 if was_modified
168     fprintf('Safety limits active. Moved to: %.1f, %.1f, %.1f\n', final_x, final_y, final_z);
169 end
170 end
171
172 function toggleSpin(src, ~)
173     if src.Value == 1, is_spinning = true; set(src, 'String', 'STOP'); startSpinLoop();
174     else, is_spinning = false; set(src, 'String', 'AUTO SPIN');
175     end
176 end
177
178 function startSpinLoop()
179     t = 0; speed = 0.05;
180     while is_spinning && ishandle(f)
181         t = t + speed;
182         a_base = 90 + 40 * sin(t);
183         a_shld = 82 + 15 * sin(t * 1.5 + pi/2);
184         a_elb = 90 + 30 * sin(t * 1.2);
185
186         new_angs = [a_base, a_shld, a_elb];
187         new_angs(1) = max(LIM_B(1), min(LIM_B(2), new_angs(1)));
188         new_angs(2) = max(LIM_S(1), min(LIM_S(2), new_angs(2)));
189         new_angs(3) = max(LIM_E(1), min(LIM_E(2), new_angs(3)));

```

```

190         curr_angles = new_angs;
191         sendCmd(s, curr_angles);
192         updateSystem();
193         pause(0.05); drawnow;
194     end
195 end
196
197 function handleKey(~, e)
198     if is_spinning, is_spinning=false; return; end
199     step = 2.0; new_angs = curr_angles;
200     switch e.Key
201         case 'leftarrow', new_angs(1) = new_angs(1) + step;
202         case 'rightarrow', new_angs(1) = new_angs(1) - step;
203         case 'uparrow', new_angs(2) = new_angs(2) + step;
204         case 'downarrow', new_angs(2) = new_angs(2) - step;
205         case 'w', new_angs(3) = new_angs(3) + step;
206         case 's', new_angs(3) = new_angs(3) - step;
207     end
208     new_angs(1) = max(LIM_B(1), min(LIM_B(2), new_angs(1)));
209     new_angs(2) = max(LIM_S(1), min(LIM_S(2), new_angs(2)));
210     new_angs(3) = max(LIM_E(1), min(LIM_E(2), new_angs(3)));
211     curr_angles = new_angs;
212     sendCmd(s, curr_angles);
213     updateSystem();
214 end
215
216 function goHome(~, ~)
217     is_spinning = false;
218     smoothMove(s, curr_angles, target_home, 40);
219     curr_angles = target_home;
220     updateSystem();
221 end
222
223 function [angs, valid] = solveIK_Calibrated(x, y, z, L1, L2,
224     H1, off_sh, off_el, prev_base)
225     z_prime = z - H1;
226     r = sqrt(x^2 + y^2 + z_prime^2);
227
228     if r > (L1 + L2) || r < abs(L1 - L2)
229         angs = [90,90,90]; valid = false; return;
230     end
231
232     if abs(x) < 0.1 && abs(y) < 0.1
233         theta1 = prev_base;
234     else

```

```

234         theta1 = atan2(y, x) * 180/pi;
235         if theta1 < 0, theta1 = theta1 + 360; end
236     end
237
238     num = x^2 + y^2 + z_prime^2 - L1^2 - L2^2;
239     den = 2 * L1 * L2;
240     val = max(-1, min(1, num / den));
241     phi_rad = acos(val);
242     phi_deg = phi_rad * 180/pi;
243
244     alpha = asin(z_prime / r);
245     beta = atan2(L2 * sin(phi_rad), L1 + L2 * cos(phi_rad));
246     theta2_phys_deg = (alpha + beta) * 180/pi;
247
248     theta2_servo = theta2_phys_deg - off_sh;
249     theta3_servo = 180 - phi_deg - off_el;
250
251     ang = [theta1, theta2_servo, theta3_servo];
252
253     valid = true;
254     if ang(1) < LIM_B(1) || ang(1) > LIM_B(2), valid = false;
255         end
256     if ang(2) < LIM_S(1) || ang(2) > LIM_S(2), valid = false;
257         end
258     if ang(3) < LIM_E(1) || ang(3) > LIM_E(2), valid = false;
259         end
260 end
261
262 function [tip, shoulder, elbow] = calculateFK(ang, L1, L2, H1,
263     off_sh, off_el)
264     t1 = ang(1) * pi/180;
265     t2_phys_deg = ang(2) + off_sh;
266     t2 = t2_phys_deg * pi/180;
267     phi_deg = 180 - ang(3) - off_el;
268     phi_rad = phi_deg * pi/180;
269     rel_angle = t2 - phi_rad;
270
271     shoulder = [0, 0, H1];
272     r1 = L1 * cos(t2);
273     z1 = H1 + L1 * sin(t2);
274     elbow = [r1*cos(t1), r1*sin(t1), z1];
275
276     r2 = L1 * cos(t2) + L2 * cos(rel_angle);
277     z2 = H1 + L1 * sin(t2) + L2 * sin(rel_angle);
278     tip = [r2*cos(t1), r2*sin(t1), z2];

```

```

275     end
276
277     function updateSystem()
278         [abs_tip, abs_sh, abs_el] = calculateFK(curr_angles, L1, L2,
279             H1, OFFSET_SHOULDER, OFFSET_ELBOW);
280
281         vec_x = abs_tip(1);
282         vec_y = abs_tip(2);
283         vec_z = abs_tip(3);
284         mag = sqrt(vec_x^2 + vec_y^2 + vec_z^2);
285
286         set(t_vec_x, 'String', sprintf('X: %.1f', vec_x));
287         set(t_vec_y, 'String', sprintf('Y: %.1f', vec_y));
288         set(t_vec_z, 'String', sprintf('Z: %.1f', vec_z));
289         set(t_mag, 'String', sprintf('%.2f cm', mag));
290         set(t_ang, 'String', sprintf('[%d, %d, %d]',
291             round(curr_angles)));
292
293         p_base = [0 0 0];
294         set(h_link0, 'XData', [p_base(1) p_base(1)], 'YData',
295             [p_base(2) p_base(2)], 'ZData', [p_base(3) abs_sh(3)]);
296         set(h_link1, 'XData', [p_base(1) abs_el(1)], 'YData',
297             [p_base(2) abs_el(2)], 'ZData', [abs_sh(3) abs_el(3)]);
298         set(h_link2, 'XData', [abs_el(1) abs_tip(1)], 'YData',
299             [abs_el(2) abs_tip(2)], 'ZData', [abs_el(3) abs_tip(3)]);
300     end
301
302     function smoothMove(serialObj, start_a, end_a, steps)
303         for t = linspace(0, 1, steps)
304             fr = start_a + (end_a - start_a) * t;
305             sendCmd(serialObj, fr);
306             pause(0.02);
307         end
308     end
309
310     function sendCmd(serialObj, angs)
311         if ~isempty(serialObj)
312             try
313                 writeline(serialObj, sprintf('B %d', round(angs(1))));
314                 writeline(serialObj, sprintf('S %d', round(angs(2))));
315                 writeline(serialObj, sprintf('E %d', round(angs(3))));
316             catch
317             end
318         end
319     end

```

```
315
316     function cleanUp(~,~)
317         try delete(s); catch; end; delete(f);
318     end
319 end
```

B Arduino Firmware Code

Listing 2: Arduino Firmware for Serial Communication

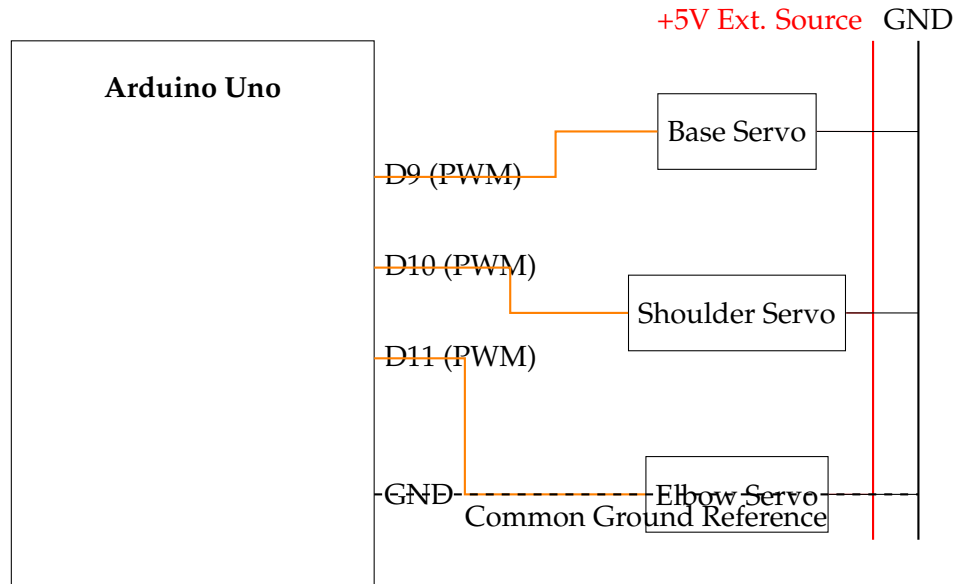
```
1  #include <Servo.h>
2
3  const int PIN_BASE    = 9;
4  const int PIN_SHOULDER = 10;
5  const int PIN_ELBOW   = 11;
6
7  Servo baseServo;
8  Servo shoulderServo;
9  Servo elbowServo;
10
11 const int BASE_MIN = 0, BASE_MAX = 180;
12 const int SHLD_MIN = 10, SHLD_MAX = 170;
13 const int ELBW_MIN = 10, ELBW_MAX = 170;
14
15 int targetBase    = 90;
16 int targetShoulder = 90;
17 int targetElbow   = 90;
18
19 String inputString = "";
20 bool stringComplete = false;
21
22 void setup() {
23     Serial.begin(9600);
24     inputString.reserve(200);
25
26     baseServo.write(targetBase);
27     shoulderServo.write(targetShoulder);
28     elbowServo.write(targetElbow);
29
30     delay(100);
31     baseServo.attach(PIN_BASE);
32     shoulderServo.attach(PIN_SHOULDER);
```

```

33     elbowServo.attach(PIN_ELBOU);
34
35     Serial.println("READY");
36 }
37
38 void loop() {
39     if (stringComplete) {
40         parseCommand(inputString);
41         inputString = "";
42         stringComplete = false;
43     }
44 }
45
46 void serialEvent() {
47     while (Serial.available()) {
48         char inChar = (char)Serial.read();
49         inputString += inChar;
50         if (inChar == '\n') {
51             stringComplete = true;
52         }
53     }
54 }
55
56 void parseCommand(String command) {
57     int bIndex = command.indexOf('B');
58     int sIndex = command.indexOf('S');
59     int eIndex = command.indexOf('E');
60
61     if (bIndex >= 0 && sIndex >= 0 && eIndex >= 0) {
62         int valBase = command.substring(bIndex + 1, sIndex).toInt();
63         int valShoulder = command.substring(sIndex + 1, eIndex).toInt();
64         int valElbow = command.substring(eIndex + 1).toInt();
65
66         moveArm(valBase, valShoulder, valElbow);
67     }
68 }
69
70 void moveArm(int b, int s, int e) {
71     targetBase = constrain(b, BASE_MIN, BASE_MAX);
72     targetShoulder = constrain(s, SHLD_MIN, SHLD_MAX);
73     targetElbow = constrain(e, ELBW_MIN, ELBW_MAX);
74
75     baseServo.write(targetBase);
76     shoulderServo.write(targetShoulder);
77     elbowServo.write(targetElbow);

```


C Circuit connections



D Bill of Materials

Component	Specification	Qty	Role
Micro Servo	TowerPro SG90 (1.8 kg-cm)	3	Actuation
Microcontroller	Arduino Uno R3 (ATmega328P)	1	Control Logic
Structural Material	3mm Birch Plywood	1 Sheet	Truss Linkages
Fasteners	M Bolts (5mm) & Safety Pins	12	Joint Assembly
Power Supply	(Breadboard) 5V DC Regulated (2A)	1	Motor Power
Software	MATLAB R2024b + Arduino Support	-	Control Interface

Table 1: System Component List

Arduino script was uploaded via VSCode with the Arduino Community Edition extension.

E Parts CAD and DXF schematics

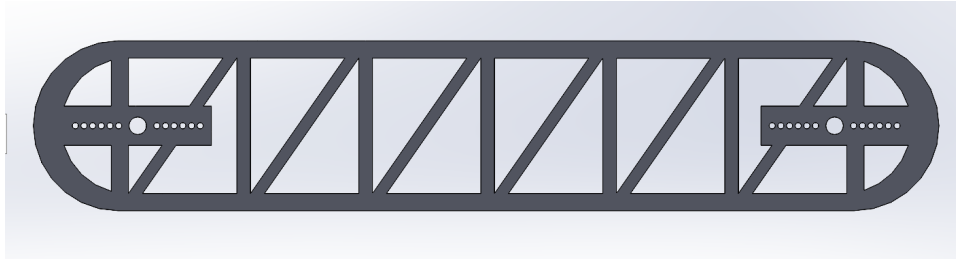


Figure 9: arm front view - CAD

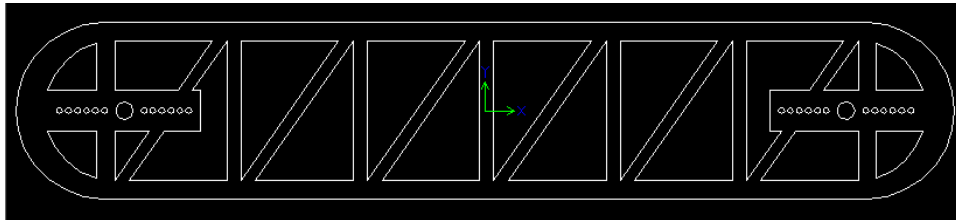


Figure 10: DXF pathlined for arm

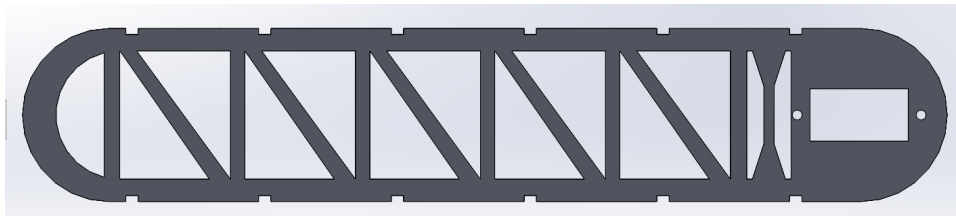


Figure 11: Forearm CAD drawing - front view

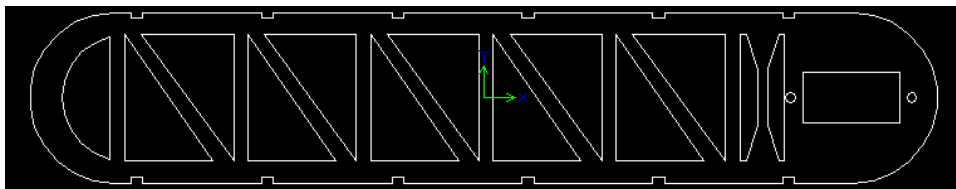


Figure 12: DXF pathlines for forearm

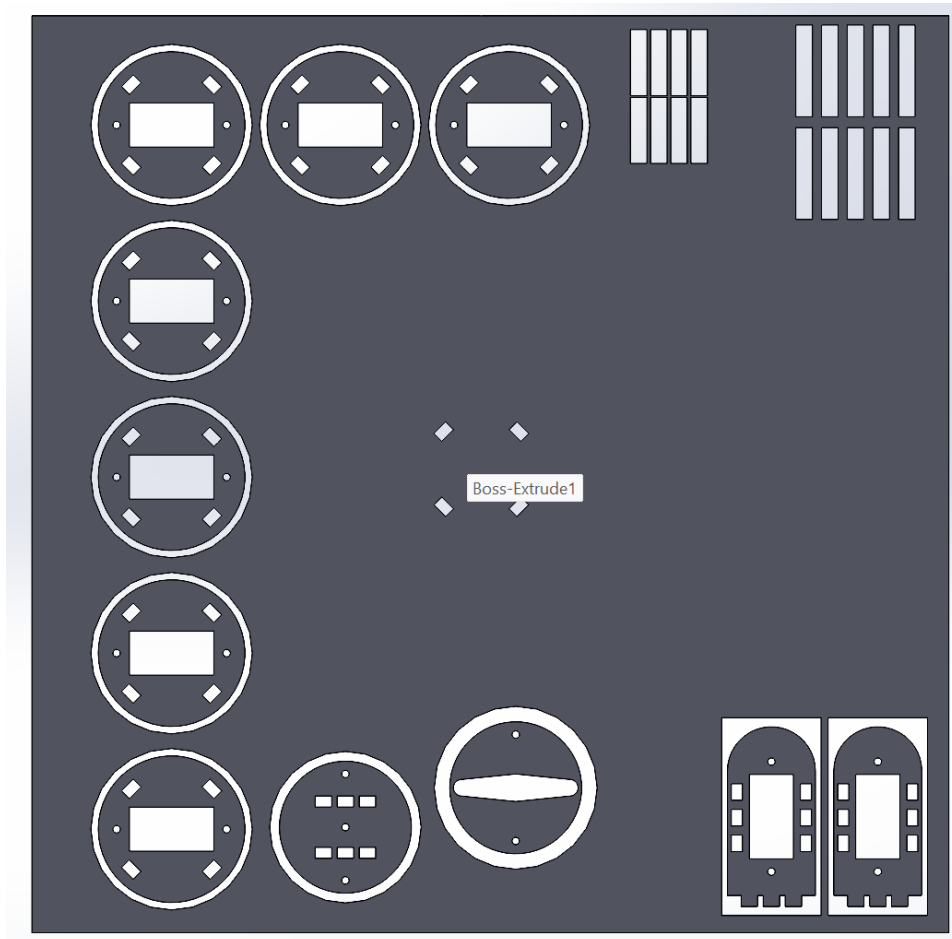


Figure 13: Baseplate in CAD, meant for the DXF file for cutting

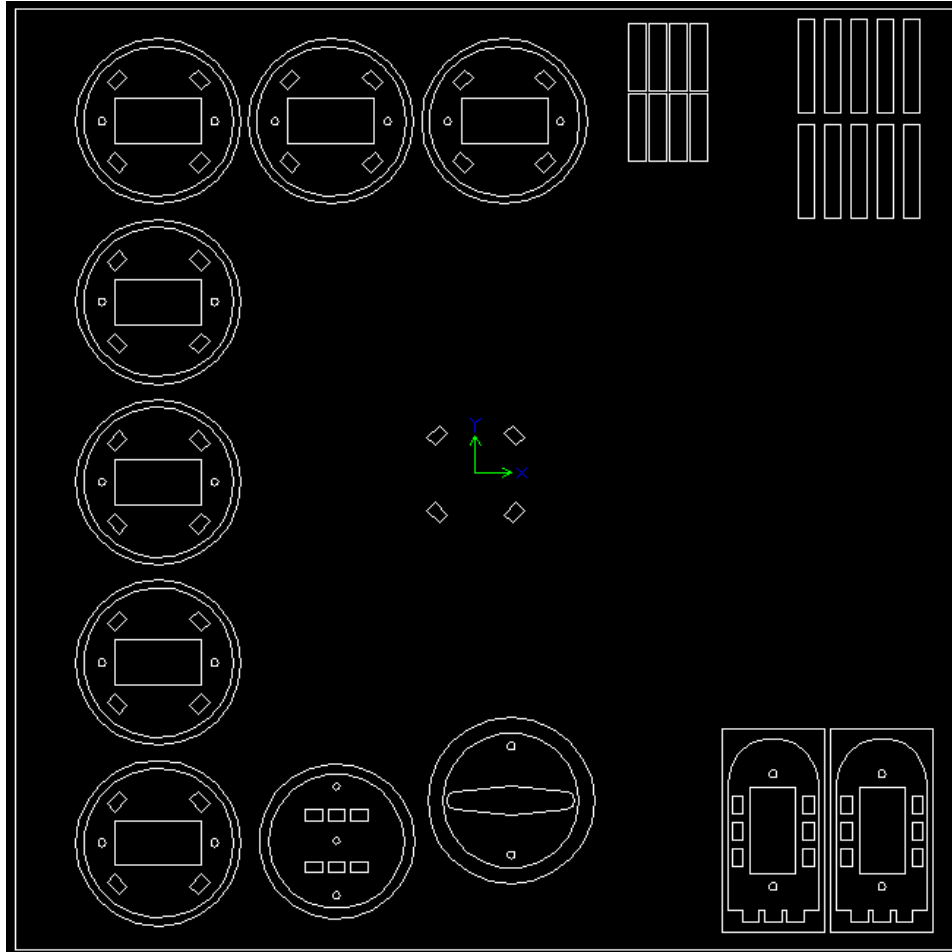


Figure 14: DXF pathlines for baseplate. The extra circle around the circular structures was made because performing an extruded cut on one circle removed all the geometry within, making us need another circle to prevent that.